

Client distribution

Which library an application needs, why there are four names, and the one file no installer can update.

An application reaches SD Core through a client DLL. **You do not have to find it inside an installed SD system** — the clients are packaged separately, with their own installers, because the person writing an application and the person running the server are usually not the same person.

The four names, and why

Two builds, each producing its DLL under **two names from one source in one build**:

BUILD	NAMES	FOR
64-bit	<code>sdclilib.dll</code> · <code>sdclient.dll</code>	new work, and existing SD applications
32-bit	<code>qmclilib.dll</code> · <code>qmclient.dll</code>	QM applications, and mvDeveloper

The `*clilib` names are what existing applications ask for and they never move. `qmclilib.dll` in particular is the original QMClient library name — it is **how an unmodified QM application finds its client at all**, so it is a name rather than a label and it is not going to be renamed.

The `*client` names are for new work and are what the installers put on `PATH`.

They are second **links**, not file copies. An import library records the DLL name its symbols come from, so a renamed copy would send the application back to the original.

Both builds come from one source

`sd4windows\sdb_ai\sd64\gplsrc\sdclilib` is the single source of truth. The 32-bit build points straight at it.

There is deliberately no second copy. An earlier build of this port had a middle hop, and it let the 32-bit DLL go stale and **ship without SCRAM** — sending passwords in clear against a server that no longer accepted them. The hop was removed.

If you are testing the 32-bit client, confirm which build you have. A client that predates SCRAM is refused outright by this server, and the refusal reads as a wrong password.

Which one does my application need?

A 64-bit application	<code>sdclient.dll</code> , or <code>sdclilib.dll</code> if it already asks for that name
----------------------	---

A 32-bit application	<code>qmclient.dll</code> , or <code>qmcclilib.dll</code> if it already asks for that name
----------------------	--

mvDeveloper	<code>qmcclilib.dll</code> , 32-bit — and see the warning below
--------------------	---

The architecture must match the application, not the machine. A 32-bit application on 64-bit Windows needs the 32-bit DLL. **The 32-bit build is a shipping deliverable, not a testing convenience.**

The one file no installer can update

mvDeveloper is free, and is a 32-bit application — <https://www.brianleach.co.uk/mvDeveloper>.

It loads its own copy of the client, from:

```
C:\Program Files (x86)\BLC\mvDeveloper\qmcclilib.dll
```

That is **beside the executable, and Windows searches an executable's own directory before** `PATH` — so no installer entry and no `PATH` change will ever update it. It has to be replaced by hand.

This is the most likely way to test an old client without realising it. If mvDeveloper cannot log in after an upgrade, check that file's date before anything else.

The library must match the release

The cleartext API login is gone. A client that still sends a password in clear is refused with *"Cleartext login is no longer supported; this server requires SCRAM authentication"*.

Two things to do, and the second is the one people miss:

1. Use a client library from this release or later.
2. **Run** `modify.password` **again for every account that uses the API.** The stored credentials changed shape and the old ones cannot be converted — the password was never kept anywhere, by design, so there is nothing to convert them from.

An account whose password has not been re-set is refused, **and the refusal reads as a wrong password**, because from the server's point of view there is no credential to check.

Connecting

Your application code does not change. `SDConnect()` and `SDConnectLocal()` take the same arguments and return the same things they always did.

<code>SDConnect()</code>	over the network, to port 4243 — not an ssh tunnel any more
<code>SDConnectLocal()</code>	on the same machine. Sends no password and never did , so SCRAM does not apply

BASIC programs reaching another SD server use the `!sdclient` class, which speaks the new login too — `connect()` takes the same arguments, but **the program has to be running under this release at both ends**.

Everything about what a connected session may open, the three gates it must clear, and the identity it runs as is on API access.

Building from source

There is one repository and it builds all four DLLs: <https://github.com/dmontaine/sd4windows>. The client source is in it, at `sdb_ai/sd64/gplsrc/sdclilib`, and `make sd` produces the 64-bit and 32-bit pairs together with the server. **No built DLL is committed**, so a clone builds — the same no-binaries rule the whole repository follows.

WHERE THE BUILT CLIENTS ARE AFTER AN INSTALL

The server installer places them under `C:\Program Files\SD`:

<code>usr\clients\client64\</code>	<code>sdclilib.dll</code> , <code>sdclient.dll</code> , and the import libraries to link against
<code>usr\clients\client32\</code>	<code>qmclilib.dll</code> , <code>qmclient.dll</code> , and their import libraries
<code>usr\bin\</code>	both DLL pairs again, beside <code>sd.exe</code> . That directory is on <code>PATH</code> , so it is where a 32-bit utility finds its client at run time

The C headers are not shipped yet. Take them from the repository.

A note on how the 32-bit client is built, because it constrains changes to it: the DLL must stay a **single self-contained file that can be copied next to an application** — hence static linking of the compiler runtime, and hence a preference for Windows' own `bcrypt.dll` and `crypt32.dll` over third-party crypto libraries. Any change to the client has to keep working in a 32-bit process.

How the clients will be published

THERE WILL BE A CLIENT INSTALLER OF ITS OWN, AND IT WILL CARRY MORE THAN THE DLLs.

In it	the client DLLs, this documentation, and the related utilities
Not in it	source code of any kind
Where the source is	GitHub only — https://github.com/dmontaine/sd4windows . The installer creates a <code>docs</code> subdirectory and names the repository there

That installer is not what W1.0-0 ships. It is a change to the installer and it has not been made yet. What exists today is what the section above describes: the server installer builds and places all four DLLs, and there is no client-only package.

So if you need a client and cannot find one, ask rather than building from source — the answer may be that a package is waiting.